Senthen Velmurugan

Dallas, TX · Open to relocation anywhere in the US (Pacific Northwest / Texas preferred)
+1 (832) 402 - 0839 senthenvelmurugan@gmail.com sent-hen.github.io

Education

Georgia Institute of Technology **In Progress**
Master of Science in Computer Science (OMSCS) *Atlanta, GA*

The University of Texas at Dallas **Completed**
Bachelor of Science in Computer Science *Dallas, TX*

Experience

KloudGin **May 2024 – August 2024**
Machine Learning Engineering Intern *Remote*

- ◇ Raised first-contact resolution 20% over 12 weeks of use by taking the Generative AI team's open question, whether RAG could speed up customer case resolution, from proposal to production rollout
- ◇ Gave support engineers suggested next actions from similar past cases and accepted forum solutions by building a multi-source RAG system: an ingestion pipeline, hybrid vector + BM25 retrieval, and cross-encoder re-ranking
- ◇ Chose the retrieval design with data by framing baselines first, then comparing chunking strategies, embedding models, a BM25 fallback, and cross-encoder re-rankers
- ◇ Raised top-k precision by measuring retrieval offline and through an A/B feedback loop with support agents, then diagnosing an overfit to long threads and retuning chunking and metadata filters
- ◇ Kept suggestion quality measurable with a light evaluation harness of offline retrieval-quality metrics and spot checks of helpfulness
- ◇ Prepared the ticketing corpus and internal forum for retrieval by building document normalization, chunking, and metadata tagging into the ingestion pipeline
- ◇ Let other teams integrate it safely with tenant-scoped search, PII scrubbing, safe prompting, and a documented API
- ◇ Carried the work from proof of concept to rollout by writing design notes and working directly with stakeholders throughout

GBC Computers **May 2023 – August 2023**
Cybersecurity and Systems Intern *Mabank, TX*

- ◇ Improved client network performance by monitoring with PRTG and tracing bottlenecks in Wireshark captures
- ◇ Hardened client infrastructure with pfSense firewall rules, Snort intrusion detection, and endpoint protection
- ◇ Found and patched critical risks with Nessus vulnerability assessments, validated by OpenVAS and manual audits

Projects

P&ID Digitizer: Scanned Drawings to Graphs | *Python, PyTorch, OpenCV, FastAPI, Docker* **2026 – In Progress**

- ◇ Turned scanned P&IDs into queryable graphs (GraphML, DEXPI 2.0.1 XML) at 0.915 edge F1 and 0.997 symbol mAP@0.5 on a 75-sheet test split by chaining symbol detection, line extraction, and graph assembly
- ◇ Found small symbols on 7,000 px sheets by fine-tuning a Faster R-CNN on overlapping 1024^2 tiles, merged across seams
- ◇ Kept training cheap, under 2 GPU-hours on a laptop RTX 4070, by fine-tuning a torchvision detector on tiles with 256 px overlap
- ◇ Traced 80% of the remaining misses to nodes 2–4 px off rather than topology errors through error analysis, and reported the real-drawing gap openly (0.49 edge F1 on 8 real sheets)
- ◇ Kept the evaluation honest by running each test split exactly once and recording 26 design decisions as they were made
- ◇ Sent only doubtful edges to a person by calibrating per-edge confidence for a review UI, shipped as a FastAPI job service in a non-root Docker image with strict mypy and 99 pytest cases in CI
- ◇ Made the service reliable to run with SQLite job state and timeouts, in a CPU image that runs end to end with `make demo`
- ◇ Limited what a bad upload or model file can do with upload size and type limits and a SHA-1-checked model

DEXPI P&ID Viewer | *JavaScript, Python, XML, SVG* **2026**

- ◇ Let anyone open a DEXPI 2.0.1 plant model with nothing installed or uploaded by building a zero-dependency, single-page viewer that lays out diagrams from topology alone, with no stored coordinates
- ◇ Drew diagrams with no stored coordinates by building a Sugiyama-style layered flow layout, a stress-majorization network layout, and an auto mode that switches per connected group
- ◇ Caught invalid files before downstream tools do by validating structure and the metamodel in the browser and a CLI
- ◇ Checked models against the Core and Plant 2.0.1 specs (class existence, property kinds, target types, multiplicity bounds) by generating a compact class table from the spec XML with a Python build script
- ◇ Gave data pipelines a headless path with a Python CLI validator (lxml) that runs both XSD and metamodel checks
- ◇ Made large models navigable with equipment, valve, and instrument symbols, a reference inspector, a model tree, ID and tag search, and SVG export

Word2Eq: Seq2Seq Training & Serving | *Python, PyTorch, FastAPI, Docker, Prometheus* **2024**

- ◇ Took valid equations from 97.4% to 100% on 1,000 SVAMP problems by rebuilding a non-functional prototype into a train, evaluate, and serve platform for a 6.4M-parameter Transformer with grammar-constrained decoding
- ◇ Eliminated malformed outputs by masking logits with a pushdown-automaton grammar during decoding, then checking every answer symbolically before returning it
- ◇ Made every model traceable to its inputs with versioned artifacts whose model cards record a config fingerprint, SHA-256 data hashes, and the git revision
- ◇ Made resumed training bit-identical by checkpointing optimizer, scheduler, RNG, and in-epoch position atomically on SIGTERM, verified by 32 pytest cases in CI
- ◇ Served 150 requests/s at 73 ms p95 on one laptop CPU by micro-batching behind a bounded queue that sheds overload with a fast 503, and made training preemption-safe with bit-identical resumes
- ◇ Guarded releases with a CI promotion gate on accuracy and validity, and ran the service against availability, latency, and answer-integrity SLOs with burn-rate alerts, a runbook, and a Kubernetes manifest with probes, a PDB, and an HPA

Campbell Woodworks: Storefront for a Family Woodshop | *Jekyll, Liquid, HTML, CSS, Python* **2026**

- ◇ Put a family woodshop's work, once only in Facebook posts behind a login, in front of buyers at no running cost
- ◇ Covered 23 pieces with one Markdown file each, and call buttons instead of a cart, since the shop sells by phone
- ◇ Generated the shop, gallery, and detail pages, with category filters, from those files, published as a static Jekyll site on GitHub Pages so there is nothing for the shop to run

Thai Sizzle: Website for a Family Food Truck | *Jekyll, Liquid, HTML, CSS, Python* **2026**

- ◇ Put a Thai food truck's menu, prices, hours, and location online, with all 13 dishes matched to its menu board
- ◇ Made it owner-editable with one data file per section, plus a weekly GitHub Action that refreshes the Google rating
- ◇ Made it ready to grow: one config link turns every call button into online ordering through Squarespace, and one setting switches its Google structured data to a restaurant
- ◇ Kept the Google Places API key out of the browser by using it only at build time, as a GitHub Actions secret

Additional Projects

Metal-to-Pod: Self-Hosted CI/CD to Kubernetes | *Kubernetes, Bash, Go, Docker, Jenkins* **2026**

- ◇ Took every commit to a running pod with no cloud services on 4 VMware VMs (16 GB laptop) by building a kubeadm cluster with default-deny network policy, a git hook into Jenkins as code, and a private TLS registry
- ◇ Made releases safe to automate by gating on unit tests, shipping ∼10 MB distroless non-root images tagged by commit, and rolling back automatically when an external smoke test fails
- ◇ Ran a Go delivery-promise service against 99.5% availability and 300 ms latency SLOs, with burn-rate alerts unit-tested in promtool, 8 failure drills, and 5 runbooks
- ◇ Made the cluster reproducible from one env file with Vagrant and shell, on containerd and Calico, by fixing the dual-NIC node IP and CNI interface issues explicitly
- ◇ Limited the blast radius of a bad deploy with namespaces, Pod Security, quotas, and a namespace-scoped ServiceAccount instead of cluster-admin

Personal VPN | *Terraform, Go, Bash, WireGuard, Linux* **2026**

- ◇ Kept traffic on untrusted networks private without trusting a commercial VPN by provisioning a hardened WireGuard server on Hetzner for about €4/month with one `terraform apply`
- ◇ Let devices join or leave without rebuilding the server by writing a `vpn-peer` tool, plus a manual runbook that mirrors every automated step
- ◇ Kept every private key off the network and out of Terraform state by generating keys on each device and the server key on first boot, managed with `vpnctl`, a Go CLI with 11 unit tests
- ◇ Cut the attack surface to one UDP port with default-deny nftables and cloud firewall rules, and documented a threat model of what it does and does not protect
- ◇ Hardened the server from first boot with cloud-init SSH hardening, key-only SSH limited to listed admin CIDRs, fail2ban, and unattended security upgrades

The $2 Billion Patch Note: Market Event Study | *Python, pandas, NumPy, SciPy, Matplotlib* **2026**

- ◇ Measured how Counter-Strike 2's October 2025 trade-up update repriced about 3,900 items (a 19% drawdown, trough at 25 hours) by building price indices from daily Steam market medians
- ◇ Showed newly useful Covert inputs gained 43.5% mean abnormal return within 48 hours ($t = 8.95$) with a constant-mean event study by rarity and wear, in a tested Python CLI that regenerates every figure
- ◇ Found the cheapest exteriors repriced most (Battle-Scarred Coverts up 258% at 14 days, Factory New down 24%) by splitting abnormal returns by wear and StatTrak
- ◇ Priced the new contract as a call option, with five Coverts as the strike and a knife from the collection's pool as the payoff, using expected-value, Monte Carlo, and stress-test views

Personal Website: sent-hen.github.io | *Jekyll, Liquid, HTML, CSS, JavaScript* **2023**

- ◇ Published a personal site with nothing to run or maintain as a static Jekyll site on GitHub Pages, where every list is one data file, so adding a project is a one-file change
- ◇ Drew its animations (spinning coin, 0/1 rain, Sierpinski triangle, ripple) as generated SVG and plain JavaScript with no libraries, with a Hi/Lo switch for still line art

The Thousand and One Words: SAT Vocabulary Comic Epic | *Python, HTML, SVG* **2026**

- ◇ Made 1,000 SAT words learnable as one story by writing a 100-chapter comic epic for readers aged 14 to 16, ten words a chapter, each with a story scene, a picture link, and a memory-palace station
- ◇ Generated all 100 chapter pages, image prompts, and the combined script from one Python source, so a one-line edit updates every page
- ◇ Linked all 1,000 words into one unbroken chain across chapters, on a memory palace of 100 landmarks in the Round City of Baghdad with the same 10 stations each

Technical Skills

Languages: Python, JavaScript, Go, C/C++, SQL, Bash, HTML, CSS, LaTeX
Frameworks: PyTorch, FastAPI, LangChain, OpenCV, pandas, NumPy, SciPy, XGBoost, Gradio, NetworkX, pytest, Jekyll
Tools: Linux (Debian), Git, Docker, Kubernetes, Terraform, PostgreSQL, Pinecone, CI/CD, Autodesk Fusion, Jenkins, Prometheus, AWS, GitHub Actions, WireGuard, nftables, pfSense, Snort, Nessus, Wireshark

Organizations

Artificial Intelligence Society **January 2024 – May 2024**

Semester Project Mentorship

- ◇ Cut prediction error 15% in product pricing forecasts by designing and implementing XGBoost and SARIMAX models, with feature selection and hyperparameter tuning
- ◇ Delivered a Gradio-based predictive analytics web application by mentoring a team of 5 underclassmen through weekly meetings, technical guidance, and code reviews

Mensa **2013 – Present**